

FAST MODEL PREDICTIVE CONTROL OF MINIATURE HELICOPTERS

FINAL PROJECT FOR THE COURSE

“MODELING AND CONTROL OF CYBER-PHYSICAL SYSTEMS II” – UNITS

MATTEO TAVANO

MSc Student in Data Science and Artificial Intelligence

Università degli Studi di Trieste

GitHub

`matteo.tavano@studenti.units.it`

Contents

I	Introduction	2
1	Introduction	2
II	System Modeling and Planning	3
2	Helicopter Model	3
2.1	Nonlinear Continuous-Time Dynamics	3
2.2	Differential Flatness and Inverse Dynamics	4
2.2.1	State Reconstruction	5
2.2.2	Input Reconstruction	5
2.3	Linear Time Varying Model	5
2.3.1	Model Simplification	6
2.4	Discretization	7
3	Trajectory Generation	7
3.1	Differential Flatness and Flat Outputs	8
3.2	Trajectory Abstraction and Implementation	8
3.3	Hover	8
3.4	Circular Trajectory	9
3.5	Spiral (Ellipsoidal Helix) Trajectory	10
3.6	Remarks on Feasibility	10
III	Control Methods	11
4	Model Predictive Control	11
4.1	LTV-MPC Formulation	11
4.2	Controller Implementation	12
4.2.1	Linearization and Warm Start	12
4.2.2	Sparse QP Formulation	13
4.3	Terminal Cost and Terminal Set	14
4.3.1	Hover: Static Equilibrium Linearization	14
4.3.2	Circle and Spiral: Adaptive Reference Linearization	14
4.3.3	Stability Verification	14

5	Proportional Integral Derivative	15
5.1	Control Architecture	15
5.1.1	Error Computation and Shortest-Path Logic	15
5.1.2	Inertial-to-Body Transformation	15
5.2	Control Law Implementation	15
5.2.1	Integral Accumulation and Anti-Windup	16
5.2.2	Discrete Derivative and Gravity Feedforward	16
5.2.3	Final Actuator Command	16
IV	Results and Discussion	16
6	Experimental Results and Analysis of MPC	16
6.1	Simulation Method and PW Constant Inputs	17
6.2	Sampling Time and Initial Condition Impact	17
6.2.1	Circle Trajectory Results	18
6.2.2	Spiral Trajectory Results	19
6.2.3	Position Error Analysis	21
6.2.4	Linearization Error Analysis	23
7	Comparative Study: MPC against PID	24
7.1	Comparison Criteria	24
7.2	Evaluation Metrics	24
7.3	MPC Configurations	25
7.3.1	Paper-Fidelity Configuration	25
7.3.2	High-Performance Configuration	25
7.3.3	Terminal Cost Configuration	26
7.4	PID Tuning Strategies	26
7.4.1	Standard Tuning	26
7.4.2	High-Performance Tuning	26
7.5	MPC Results	27
7.5.1	Circle Trajectory	27
7.5.2	Spiral Trajectory	28
7.5.3	Temporal Tracking Performance	29
7.6	PID Results	30
7.6.1	Circle Trajectory	30
7.6.2	Spiral Trajectory	31
7.7	Overall Comparison	31
7.7.1	Trajectory Tracking Performance	31
7.7.2	Comparing Controller Performance	32
7.8	Impact of Terminal Cost	32
7.8.1	Analysis of Tracking Performance Trade-offs	33
V	Conclusions	33

This work presents a MATLAB based replication and extension of the control framework proposed by the authors for Fast Model Predictive Control of Miniature Helicopters. The original approach is reproduced with the objective of validating the reported results, while introducing improvements in the implementation and extending the analysis to additional scenarios. Trajectory generation is performed following the method proposed by Murray, ensuring dynamically feasible reference trajectories that respect the system constraints. The helicopter dynamics are modeled and discretized for real-time control, and a fast Model Predictive Controller (MPC) is implemented and evaluated. Several scenarios are tested, using different geometric trajectories, different initial conditions and sampling times. For comparison purposes, a classical Proportional Integral Derivative (PID) controller is also considered. Simulation results demonstrate that the replicated framework successfully reproduces the behavior described by the authors, while the proposed extensions improve tracking performance and robustness, confirming the effectiveness of fast MPC for aerial systems.

Introduction

SECTION 1

Introduction

Miniature helicopters represent a challenging class of aerial vehicles due to their highly nonlinear dynamics, strong coupling between translational and rotational motion, and fast time scales. Their reduced mass and inertia make them particularly sensitive to model uncertainties, external disturbances, and sensor noise, while the limited actuation intensiveness imposes strict constraints on admissible control inputs. These characteristics make the design of reliable control and estimation strategies for miniature helicopters a nontrivial task, especially when accurate trajectory tracking and real-time implementation are required.

Model Predictive Control (MPC) has emerged as a powerful control paradigm for constrained multivariable systems, as it explicitly accounts for system dynamics, input and state constraints, and performance objectives within an optimization-based framework. Traditionally, MPC has been associated with slow industrial processes due to the computational bottleneck of solving an optimization problem online. However, recent advances in computational hardware and fast convex optimization techniques have enabled the application of MPC to fast dynamical systems, including small-scale aerial vehicles with sampling times in the order of milliseconds (ms). In this context, the work by Kunz et al. [1] demonstrated that fast MPC can be successfully applied to miniature helicopters by exploiting model simplifications, **Linear Time Varying (LTV)** approximations, and tailored optimization solvers.

The objective of this project is to replicate, validate, and extend the control framework proposed by the authors using a MATLAB based implementation. The entire control and estimation architecture is developed in MATLAB R2024b, with the goal of reproducing the main results reported in the original work while providing a transparent and modular implementation suitable for further experimentation. In addition to replication, this work aims to improve certain aspects of the original approach and extend the analysis to additional simulation scenarios, including different sampling times and placements of the helicopter.

Trajectory generation plays a key role in the proposed framework. In particular, dynamically feasible reference trajectories are generated offline using the **differential flatness-based method** introduced by Murray et al. [2], ensuring consistency with the nonlinear helicopter dynamics. These trajectories are subsequently tracked using a fast MPC controller based on an LTV approximation of the system model. For benchmarking purposes, a classical **Proportional Integral Derivative (PID)** controller is also implemented and tuned to provide a baseline for performance comparison.

The remainder of this report is organized as follows. Part II presents the helicopter model and the trajectory generation method and adopted in this work. Part III introduces the MPC formulation and the PID controller used for comparison. Part IV discusses the experimental and simulation results, focusing specifically on various aspects of MPC implementation and highlighting performance differences between control strategies. Finally, Part V concludes the report and outlines possible directions for

future work.

System Modeling and Planning

PART

II

SECTION 2

Helicopter Model

This section details the mathematical modeling of the miniature helicopter, covering the nonlinear continuous-time dynamics, the differential flatness mapping used for trajectory generation, and the linearization and discretization steps required for the MPC formulation.

SUBSECTION 2.1

Nonlinear Continuous-Time Dynamics

The helicopter is modeled as a rigid body with six degrees of freedom, augmented with integral states to ensure zero steady-state error during tracking. The state vector $\mathbf{x} \in \mathbb{R}^{10}$ and the control input vector $\mathbf{u} \in \mathbb{R}^4$ are defined as:

$$\mathbf{x} = [x_I \ y_I \ z_I \ \Psi \ \dot{x}_B \ \dot{y}_B \ \dot{z}_B \ \dot{\Psi} \ x_{int} \ y_{int}]^T, \quad \mathbf{u} = [u_x \ u_y \ u_z \ u_\Psi]^T, \quad (2.1)$$

where:

- $[x_I, y_I, z_I]^T$ is the position in the inertial frame;
- Ψ is the yaw angle;
- $[\dot{x}_B, \dot{y}_B, \dot{z}_B]^T$ are the velocities in the body frame;
- $\dot{\Psi}$ is the yaw rate;
- x_{int}, y_{int} are the integral error states for the x and y positions.

The *inertial frame* is a fixed global coordinate system used to measure the absolute position and orientation of the helicopter within the navigation space. In contrast, the *body frame* refers to the vehicle's center of gravity. The inputs u_x, u_y, u_z, u_Ψ represent the commands for pitch, roll, thrust, and yaw rate, respectively. Based on the implementation in `HelicopterModel.m`, the nonlinear dynamics are described by the following set of differential equations:

$$\dot{x}_I = \cos(\Psi)\dot{x}_B - \sin(\Psi)\dot{y}_B, \quad (2.2)$$

$$\dot{y}_I = \sin(\Psi)\dot{x}_B + \cos(\Psi)\dot{y}_B, \quad (2.3)$$

$$\dot{z}_I = \dot{z}_B, \quad (2.4)$$

$$\dot{\Psi} = \dot{\Psi}, \quad (2.5)$$

$$\ddot{x}_B = b_x u_x + k_x \dot{x}_B + \dot{\Psi} \dot{y}_B, \quad (2.6)$$

$$\ddot{y}_B = b_y u_y + k_y \dot{y}_B - \dot{\Psi} \dot{x}_B, \quad (2.7)$$

$$\ddot{z}_B = b_z u_z - g, \quad (2.8)$$

$$\ddot{\Psi} = b_\Psi u_\Psi + k_\Psi \dot{\Psi}, \quad (2.9)$$

$$\dot{x}_{int} = k_i(x_I - x_{ref}), \quad (2.10)$$

$$\dot{y}_{int} = k_i(y_I - y_{ref}). \quad (2.11)$$

Here, $b_{(\cdot)}$ parameters represent the control effectiveness gains, $k_{(\cdot)}$ are the drag/damping coefficients, k_i is the integral parameter and g is the gravitational acceleration. The terms $\pm \dot{\Psi} \dot{x}_B$ and $+\dot{\Psi} \dot{y}_B$ in (2.6) and (2.7) represent the Coriolis and centripetal coupling effects due to the rotating body frame.

To enhance closed-loop robustness, the authors simplified the controller's prediction model by omitting Coriolis and cross-coupling terms, using the identified model parameters: $g = 9.81$, $b_x = 2.0$, $b_y = 2.1$, $b_z = 18.0$, $b_\Psi = 2.0$ and $k_x = -0.5$, $k_y = -0.4$, $k_\Psi = -5.0$, $k_i = 2.0$.

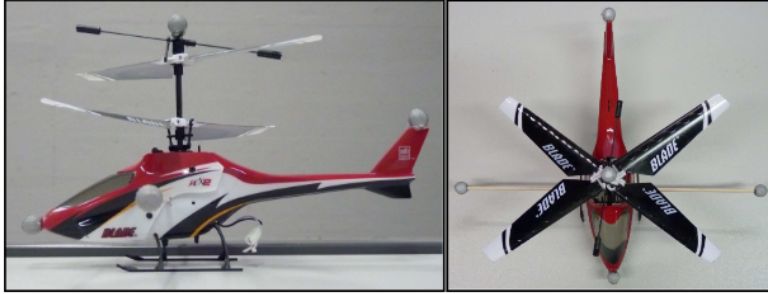


Figure 1. Blade mCX2 micro-coaxial helicopter.

SUBSECTION 2.2

Differential Flatness and Inverse Dynamics

To generate feasible reference trajectories $[\mathbf{x}_{ref}, \mathbf{u}_{ref}]$, the system exploits the property of differential flatness. As implemented in `FlatnessMap.m`, the flat outputs are selected as the inertial position and heading:

$$\mathbf{z} = [x_I, y_I, z_I, \Psi]^T. \quad (2.12)$$

Given a desired trajectory $\mathbf{z}(t)$ and its derivatives $\dot{\mathbf{z}}, \ddot{\mathbf{z}}$, the full state and input can be reconstructed algebraically.

2.2.1 State Reconstruction

The positional and yaw states are trivial mappings of $\mathbf{z}$. The body velocities are recovered by inverting the kinematic rotation matrix $R_z(\Psi)$:

$$\begin{bmatrix} \dot{x}_B \\ \dot{y}_B \end{bmatrix} = \begin{bmatrix} \cos \Psi & \sin \Psi \\ -\sin \Psi & \cos \Psi \end{bmatrix} \begin{bmatrix} \dot{x}_I \\ \dot{y}_I \end{bmatrix}. \quad (2.13)$$

Thus, the following equations are obtained:

$$\dot{x}_B = \cos \Psi \dot{x}_I + \sin \Psi \dot{y}_I, \quad (2.14)$$

$$\dot{y}_B = -\sin \Psi \dot{x}_I + \cos \Psi \dot{y}_I, \quad (2.15)$$

$$\dot{z}_B = \dot{z}_I, \quad (2.16)$$

$$\dot{\Psi} = \dot{\Psi}. \quad (2.17)$$

2.2.2 Input Reconstruction

The next step in the flatness-based trajectory generation is to determine the control inputs $\mathbf{u} = [u_x, u_y, u_z, u_\Psi]^T$ as functions of the flat outputs and their derivatives. This is achieved by inverting the system dynamics.

For the vertical and yaw axes, the relationship is straightforward. From the dynamic equations, the thrust u_z and yaw command u_Ψ are obtained directly:

$$u_z = \frac{1}{b_z}(\ddot{z}_I + g), \quad (2.18)$$

$$u_\Psi = \frac{1}{b_\Psi}(\ddot{\Psi} - k_\Psi \dot{\Psi}). \quad (2.19)$$

For the horizontal plane, the pitch (u_x) and roll (u_y) inputs depend only on the projection of the inertial acceleration and the drag component. The resulting algebraic mappings are:

$$u_x = \frac{1}{b_x} (\cos(\Psi)[\ddot{x}_I - k_x \dot{x}_B] + \sin(\Psi)[\ddot{y}_I - k_x \dot{y}_B]), \quad (2.20)$$

$$u_y = \frac{1}{b_y} (-\sin(\Psi)[\ddot{x}_I - k_y \dot{x}_B] + \cos(\Psi)[\ddot{y}_I - k_y \dot{y}_B]), \quad (2.21)$$

where $\dot{x}_B$ and $\dot{y}_B$ are the body velocities previously computed in the state reconstruction step.

Equations (2.18)–(2.21) constitute the function $\mathbf{u}_{ref} = i(\mathbf{z}, \dot{\mathbf{z}}, \ddot{\mathbf{z}})$, allowing the offline generation of the feedforward control trajectory.

SUBSECTION 2.3

Linear Time Varying Model

To employ a fast Model Predictive Control scheme, the nonlinear dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ must be approximated by a linear model suitable for convex optimization. A Linear Time-Varying (LTV) approximation has been employed, obtained by linearizing the system dynamics around the nominal reference trajectory $(\hat{\mathbf{x}}_k, \hat{\mathbf{u}}_k)$ at each time step k . Let u be a **piecewise constant** function in time given by: $u(t) = u_k$, with $kT_s \leq t \leq (k+1)T_s$ and $k \in \mathbb{Z}^+$, where T_s is the sampling time period of the system. We will refer to $\hat{u}(t)$ as the *nominal inputs* and $\hat{x}(t)$ as the *nominal state* trajectory generated by applying the piecewise constant inputs to the nonlinear system. Starting at some time $t_0 = k_0T_s$, $k_0 \in \mathbb{Z}^+$ with initial condition $x(k_0T_s) = x_0$, the nominal state and input trajectories are sampled with:

$$\hat{x}_k = \hat{x}(kT_s), \quad (2.22)$$

$$\hat{u}_k = \hat{u}(kT_s). \quad (2.23)$$

The system $\dot{x} = f(x, u)$, is then linearized at p time instances of the nominal trajectory leading to the following set of linearized dynamics:

$$\delta \dot{\mathbf{x}}(t) = A_k \delta \mathbf{x}(t) + B_k \delta \mathbf{u}(t), \quad (2.24)$$

where $\delta \mathbf{x} = \mathbf{x} - \hat{\mathbf{x}}_k$ and $\delta \mathbf{u} = \mathbf{u} - \hat{\mathbf{u}}_k$. The Jacobian matrices are defined as:

$$A_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_k, \hat{\mathbf{u}}_k}, \quad B_k = \left. \frac{\partial f}{\partial \mathbf{u}} \right|_{\hat{\mathbf{x}}_k, \hat{\mathbf{u}}_k}. \quad (2.25)$$

Based on the dynamics derived, the state Jacobian matrix $A_k \in \mathbb{R}^{10 \times 10}$ is derived analytically. Evaluating the partial derivatives at the nominal point yields:

$$A_k = \begin{bmatrix} 0 & 0 & 0 & -\dot{y}_I & \cos \Psi & -\sin \Psi & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \dot{x}_I & \sin \Psi & \cos \Psi & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & k_x & \dot{\Psi} & 0 & \dot{y}_B & 0 & 0 \\ 0 & 0 & 0 & 0 & -\dot{\Psi} & k_y & 0 & -\dot{x}_B & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & k_\Psi & 0 & 0 \\ k_i & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & k_i & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \quad (2.26)$$

In the first two rows, the kinematic relationships $\frac{\partial \dot{x}_I}{\partial \Psi} = -\dot{y}_I$ and $\frac{\partial \dot{y}_I}{\partial \Psi} = \dot{x}_I$ are used to maintain compactness. Rows 5 and 6 capture the aerodynamic drag (k_x, k_y), the Coriolis effect ($\pm \dot{\Psi}$), and the coupling between the yaw rate and body velocities ($\pm \dot{y}_B, \pm \dot{x}_B$).

The input Jacobian matrix $B_k \in \mathbb{R}^{10 \times 4}$ maps the control inputs directly to the state derivatives:

$$B_k = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ b_x & 0 & 0 & 0 \\ 0 & b_y & 0 & 0 \\ 0 & 0 & b_z & 0 \\ 0 & 0 & 0 & b_\Psi \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \quad (2.27)$$

Remark The B_k matrix linearized around the nominal trajectory is a **Linear Time-Invariant (LTI)** Matrix.

2.3.1 Model Simplification

The analytical linearization in Eq. (2.26) reveals significant cross-coupling terms between the longitudinal and lateral dynamics via the rotational states. Specifically, these are the Coriolis terms ($\dot{\Psi}$ in columns 5 and 6) and the yaw-velocity coupling terms ($\dot{y}_B, -\dot{x}_B$ in column 8).

As noted by Kunz et al., these coupling terms can introduce errors in the prediction model when there is a model mismatch or noisy state estimation. To enhance the robustness of the closed-loop system, the authors propose a simplified prediction model where these specific off-diagonal terms are neglected. In this implementation, this strategy has been adopted by setting:

$$A_k(5, 6) = 0, \quad A_k(5, 8) = 0, \quad A_k(6, 5) = 0, \quad A_k(6, 8) = 0. \quad (2.28)$$

This simplification effectively decouples the x and y body accelerations from the yaw dynamics in the MPC prediction horizon, resulting in a more stable controller behavior. In this MATLAB implementation, this simplification is applied by setting the `simplify_coupling_flag` to `true`.

SUBSECTION 2.4

Discretization

To facilitate control, the continuous linearized system $(\mathbf{x}, \mathbf{u}, A_k, B_k)$ is discretized. The framework implements two methods in `Discretization.m`:

1. **Forward Euler:** A first-order approximation used in the original work for its low computational cost:

$$A_d = I + A_k T_s, \quad B_d = B_k T_s. \quad (2.29)$$

2. **Zero-Order Hold (ZOH):** An exact discretization assuming constant inter-sample inputs, implemented via matrix exponential for higher accuracy:

$$M = \exp \left(\begin{bmatrix} A_k & B_k \\ \mathbf{0} & \mathbf{0} \end{bmatrix} T_s \right) \implies A_d = M_{1:n, 1:n}, \quad B_d = M_{1:n, n+1:end}. \quad (2.30)$$

In the original paper and in this replica, Euler has been used as default discretization method for the experiments. These discrete matrices A_d, B_d form the equality constraints of the MPC optimization problem.

SECTION 3

Trajectory Generation

Trajectory generation is a fundamental component of the proposed control framework, as Model Predictive Control is designed to track reference trajectories that are dynamically feasible with respect to the system dynamics. In this work, the control objective is formulated as the tracking of a time-varying reference of the form:

$$[\mathbf{x}_r(t), \mathbf{u}_r(t)]^T, \quad (3.1)$$

where the reference state and input trajectories are generated offline and provided to the controller at runtime.

Reference trajectories are generated using a **differential flatness-based** approach, following the methodology introduced by Murray and adopted in the reference paper on fast MPC for miniature helicopters. This choice guarantees that the resulting trajectories are consistent with the nonlinear helicopter dynamics and can be accurately tracked by the MPC controller.

SUBSECTION 3.1

Differential Flatness and Flat Outputs

A nonlinear system is said to be *differentially flat* if there exists a set of outputs, referred to as *flat outputs*, such that all system states and control inputs can be expressed as algebraic functions of these outputs and a finite number of their time derivatives, without requiring integration.

Following the formulation adopted in the reference work, the helicopter model is flat with respect to the output vector:

$$\mathbf{z}(t) = \begin{bmatrix} z_1(t) \\ z_2(t) \\ z_3(t) \\ z_4(t) \end{bmatrix} = \begin{bmatrix} x_I(t) \\ y_I(t) \\ z_I(t) \\ \Psi(t) \end{bmatrix}, \quad (3.2)$$

where x_I, y_I, z_I denote the helicopter position in the inertial frame and Ψ is the yaw angle.

Given the flat outputs and their time derivatives,

$$\dot{\mathbf{z}}(t) = \begin{bmatrix} \dot{z}_1(t) \\ \dot{z}_2(t) \\ \dot{z}_3(t) \\ \dot{z}_4(t) \end{bmatrix}, \quad \ddot{\mathbf{z}}(t) = \begin{bmatrix} \ddot{z}_1(t) \\ \ddot{z}_2(t) \\ \ddot{z}_3(t) \\ \ddot{z}_4(t) \end{bmatrix}, \quad (3.3)$$

all remaining states and control inputs can be reconstructed through algebraic mappings of the form:

$$\mathbf{x}_r(t) = h(\mathbf{z}, \dot{\mathbf{z}}), \quad \mathbf{u}_r(t) = i(\mathbf{z}, \dot{\mathbf{z}}, \ddot{\mathbf{z}}). \quad (3.4)$$

This property reduces the trajectory generation problem to the definition of smooth time profiles for the flat outputs.

SUBSECTION 3.2

Trajectory Abstraction and Implementation

In this MATLAB implementation, reference trajectories are defined through an abstract base class `TrajectoryBase.m` that specifies a common interface for all trajectory shapes. Each trajectory provides the flat outputs and their derivatives:

$$\{\mathbf{z}(t), \dot{\mathbf{z}}(t), \ddot{\mathbf{z}}(t)\}, \quad (3.5)$$

which are subsequently mapped to reference state and input trajectories for the MPC controller. This abstraction enables modularity and allows different trajectories to be tested without modifying the control or estimation architecture.

SUBSECTION 3.3

Hover

The hover stationary flight, defined in `ShapeHover.m` corresponds to a regulation problem and is used to replicate the hovering experiments presented by the authors in the reference paper. The helicopter is required to maintain a fixed position and heading, making this scenario particularly suitable for evaluating stabilization performance and disturbance rejection.

The flat output vector and its derivatives are defined as:

$$\mathbf{z}(t) = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \\ \Psi_0 \end{bmatrix}, \quad \dot{\mathbf{z}}(t) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \ddot{\mathbf{z}}(t) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}. \quad (3.6)$$

The default parameters used in this work are

- target position $\mathbf{p}_0 = [x_0 \ y_0 \ z_0]^T = [0 \ 0 \ 1]^T$ m;
- target yaw angle $\Psi_0 = 0$ rad;
- hover duration $T_{\text{hover}} = 5.0$ s.

SUBSECTION 3.4

Circular Trajectory

The circular trajectory describes motion in the horizontal plane at constant altitude and is used to evaluate tracking performance during coordinated maneuvers. It is defined in `ShapeCircle.m`. In vector form, the flat outputs are defined as:

$$\mathbf{z}(t) = \begin{bmatrix} R \cos(\omega t) \\ R \sin(\omega t) \\ h \\ \omega t + \frac{\pi}{2} \end{bmatrix}, \quad \dot{\mathbf{z}}(t) = \begin{bmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \\ 0 \\ \omega \end{bmatrix}, \quad \ddot{\mathbf{z}}(t) = \begin{bmatrix} -R\omega^2 \cos(\omega t) \\ -R\omega^2 \sin(\omega t) \\ 0 \\ 0 \end{bmatrix}. \quad (3.7)$$

Expanding the components:

$$z_1(t) = R \cos(\omega t), \quad z_2(t) = R \sin(\omega t), \quad (3.8)$$

$$z_3(t) = h, \quad z_4(t) = \omega t + \frac{\pi}{2} \quad (3.9)$$

The default configuration parameters are:

- radius $R = 1.0$ m;
- angular velocity $\omega = 1.0$ rad/s;
- altitude $h = 1.0$ m;
- trajectory duration $T_{\text{circle}} = 2\pi$ s, which corresponds to one lap following the circle.

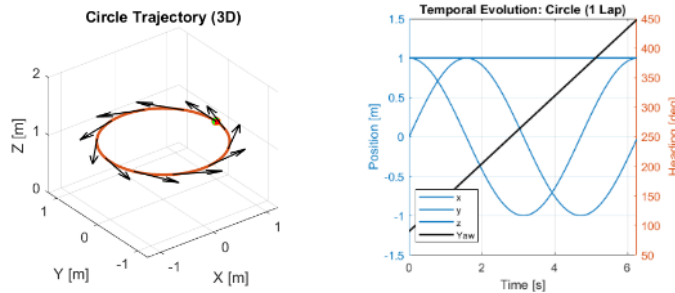


Figure 2. Visualization of the circle trajectory:
(a) Circle Trajectory in 3D space. (b) Evolution of x , y , z , and Ψ during T_{circle} .

SUBSECTION 3.5

Spiral (Ellipsoidal Helix) Trajectory

This subsection describes the implementation of the `ShapeSpiral.m` trajectory, which extends the circular motion by introducing a constant vertical climb. To maintain kinematic consistency during the 3D maneuver, the trajectory utilizes a tangent logic, where the heading angle is kept tangent to the velocity vector in the XY -plane. The flat output vector and its derivatives are defined as:

$$\mathbf{z}(t) = \begin{bmatrix} R \cos(\omega t) \\ R \sin(\omega t) \\ z_0 + v_z t \\ \omega t + \frac{\pi}{2} \end{bmatrix}, \quad \dot{\mathbf{z}}(t) = \begin{bmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \\ v_z \\ \omega \end{bmatrix}, \quad \ddot{\mathbf{z}}(t) = \begin{bmatrix} -R\omega^2 \cos(\omega t) \\ -R\omega^2 \sin(\omega t) \\ 0 \\ 0 \end{bmatrix}. \quad (3.10)$$

In component form, the reference outputs are recovered as:

$$z_1(t) = R \cos(\omega t), \quad z_2(t) = R \sin(\omega t), \quad (3.11)$$

$$z_3(t) = z_0 + v_z t, \quad z_4(t) = \omega t + \frac{\pi}{2}. \quad (3.12)$$

The parameters used for the simulation are:

- radius $R = 1.0$ m;
- angular velocity $\omega = 1.0$ rad/s;
- vertical velocity $v_z = 0.2$ m/s;
- initial altitude $z_0 = 0.5$ m;
- trajectory duration for exactly 3 full revolutions $T_{\text{spiral}} = \frac{6\pi}{\omega} \approx 18.85$ s.

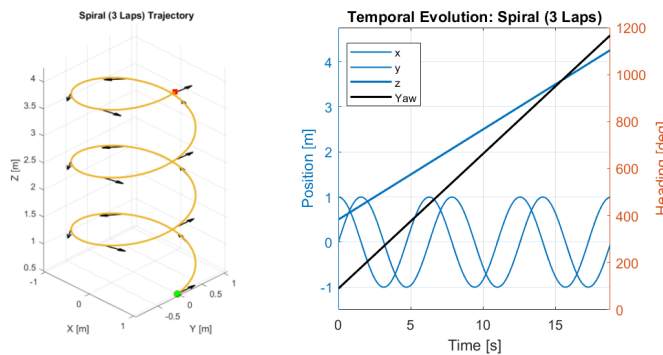


Figure 3. Visualization of the spiral trajectory:

(a) Spiral Trajectory in 3D space. (b) Evolution of x , y , z , and Ψ during T_{spiral} .

SUBSECTION 3.6

Remarks on Feasibility

All reference trajectories are generated to be smooth up to second order, ensuring continuity of accelerations and avoiding aggressive control actions. By exploiting the differential flatness property of the helicopter model, the resulting trajectories are guaranteed

to be dynamically feasible and suitable for tracking within the fast MPC framework described in the following sections.

PART

III

Control Methods

SECTION 4

Model Predictive Control

Model Predictive Control (MPC) is an optimization-based control strategy that computes the control input by solving a finite-horizon optimal control problem at each sampling instant. In this work, a fast MPC scheme based on a Linear Time-Varying (LTV) approximation of the helicopter dynamics has been implemented, enabling real-time trajectory tracking. The controller follows a receding horizon policy: at time t , the optimal input sequence is computed, the first element is applied to the system, and the optimization is repeated at $t + T_s$ with updated state measurements.

SUBSECTION 4.1

LTV-MPC Formulation

Given the current measured state $\mathbf{x}_t$ at time t and the precomputed reference trajectory $[\mathbf{x}_{ref}, \mathbf{u}_{ref}]$, the Model Predictive Control problem is formulated as a constrained **Quadratic Program (QP)**. The objective is to minimize the deviation of the predicted system trajectory from the reference over a finite prediction horizon N .

Let the decision variables be the sequence of predicted states $\mathbf{X} = [\mathbf{x}_0, \dots, \mathbf{x}_N]$ and the sequence of control inputs $\mathbf{U} = [\mathbf{u}_0, \dots, \mathbf{u}_{N-1}]$, where the index k represents the prediction step relative to the current time t (i.e., $\mathbf{x}_k \equiv \mathbf{x}_{t+k|t}$).

The optimization problem is defined as follows:

$$\min_{\mathbf{X}, \mathbf{U}} J(\mathbf{X}, \mathbf{U}) = \sum_{k=0}^{N-1} (\|\Delta \mathbf{x}_k\|_Q^2 + \|\Delta \mathbf{u}_k\|_R^2) + \|\Delta \mathbf{x}_N\|_{Q_f}^2 \quad (4.1a)$$

$$\text{subject to: } \mathbf{x}_{k+1} = A_k \mathbf{x}_k + B_k \mathbf{u}_k + \mathbf{d}_k, \quad k = 0, \dots, N-1 \quad (4.1b)$$

$$\mathbf{x}_0 = \mathbf{x}_{meas}, \quad (4.1c)$$

$$\mathbf{u}_{min} \leq \mathbf{u}_k \leq \mathbf{u}_{max}, \quad k = 0, \dots, N-1 \quad (4.1d)$$

$$\mathbf{x}_{min} \leq \mathbf{x}_k \leq \mathbf{x}_{max}, \quad k = 1, \dots, N \quad (4.1e)$$

where:

- The tracking errors are defined as $\Delta \mathbf{x}_k = \mathbf{x}_k - \mathbf{x}_{ref,k}$ and $\Delta \mathbf{u}_k = \mathbf{u}_k - \mathbf{u}_{ref,k}$.
- The term $\|\mathbf{z}\|_W^2 = \mathbf{z}^T W \mathbf{z}$ denotes the weighted squared Euclidean norm.
- $Q \succeq 0$ and $R \succ 0$ are the stage cost weighting matrices for states and inputs, respectively, and $Q_f \succeq 0$ is the terminal cost weighting matrix. These matrices ensure the convexity of the cost function.

The equality constraints (4.1b) represent the Linear Time-Varying (LTV) approximation of the nonlinear system dynamics. The matrices A_k and B_k are the Jacobians

of the system dynamics evaluated along the nominal trajectory $(\hat{\mathbf{x}}, \hat{\mathbf{u}})$. The term $\mathbf{d}_k$ is the affine linearization offset, defined as:

$$\mathbf{d}_k = f(\hat{\mathbf{x}}_k, \hat{\mathbf{u}}_k) - (A_k \hat{\mathbf{x}}_k + B_k \hat{\mathbf{u}}_k). \quad (4.2)$$

This term is essential for maintaining the accuracy of the linear prediction model.

The state constraints (4.1e) enforce the physical limits of the system states. In particular, upper and lower bounds are imposed only on the translational and yaw-rate velocities

$$\dot{\mathbf{x}} = [\dot{x}_B \ \dot{y}_B \ \dot{z}_B \ \dot{\Psi}]^T, \quad (4.3)$$

with

$$\mathbf{x}_{max} = [3 \text{ m/s} \ 3 \text{ m/s} \ 2 \text{ m/s} \ 25 \text{ rad/s}]^T, \quad \mathbf{x}_{min} = [-3 \text{ m/s} \ -3 \text{ m/s} \ -2 \text{ m/s} \ -25 \text{ rad/s}]^T. \quad (4.4)$$

Similarly, the input constraints (4.1d) are defined as box constraints derived from actuator control limits and stability considerations, given by

$$\mathbf{u}_{max} = [1 \ 1 \ 0.4 \ 1]^T, \quad \mathbf{u}_{min} = [-1 \ -1 \ -0.4 \ 0]^T. \quad (4.5)$$

These constraints define convex admissible sets $\mathcal{X}$ and $\mathcal{U}$ for the states and inputs, respectively, ensuring safe and physically feasible operation of the helicopter.

SUBSECTION 4.2

Controller Implementation

The implementation is realized in the `MPCCController.m` class, following a sparse formulation to exploit the structure of the optimal control problem for efficient solving.

4.2.1 Linearization and Warm Start

The performance of this LTV approximation heavily depends on the choice of the nominal trajectory. The authors implemented a two-stage strategy:

- **Initialization (Cold Start):** At the initial time $t = 0$, the nominal trajectory is established via two alternative strategies: either adopting the feasible reference trajectory $(\mathbf{x}_r, \mathbf{u}_r)$ generated via the differential flatness mapping or by utilizing the zero vector, corresponding to the helicopter positioned at the origin.
- **Warm Start Strategy:** For subsequent time steps, the linearization trajectory is updated via a *warm start* approach. The previously computed optimal control sequence $\mathbf{U}_t^*$ is shifted forward and its final element is duplicated to maintain the prediction horizon length p :

$$\hat{\mathbf{U}}_{t+1} = [\mathbf{u}_{t+1|t}^*, \dots, \mathbf{u}_{t+p|t}^*, \mathbf{u}_{t+p|t}^*] \quad (4.6)$$

After the linearization around the current measured states and the previously computed optimal control sequence, nominal state trajectory $\hat{\mathbf{X}}_{t+1}$ is then updated by propagating these inputs through the nonlinear model. This strategy ensures that the linearization remains valid and closely follows the actual evolution of the system, significantly improving the convergence of the solver and the tracking accuracy.

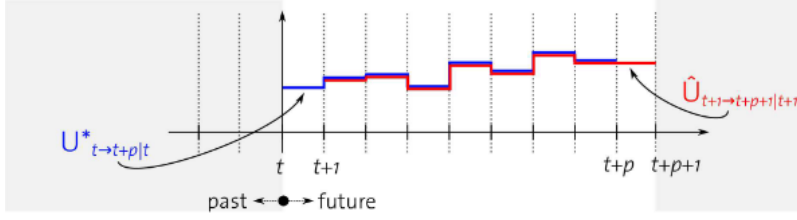


Figure 4. Visualization of the warm-start strategy: the previous optimal control sequence is shifted forward to provide a consistent nominal trajectory for the next LTV linearization.

4.2.2 Sparse QP Formulation

The LTV-MPC sub-problem is solved at each sampling instant by casting it into a standard Quadratic Programming (QP) form. To exploit the computational efficiency of modern solvers like MATLAB's `quadprog`, the optimization is structured using a **sparse formulation** where both states and inputs are treated as decision variables. The resulting optimization problem is:

$$\begin{aligned} \min_{\mathbf{Z}} \quad & \frac{1}{2} \mathbf{Z}^\top \mathbf{H} \mathbf{Z} + \mathbf{f}^\top \mathbf{Z} \\ \text{s.t.} \quad & \mathbf{A}_{eq} \mathbf{Z} = \mathbf{b}_{eq} \\ & \mathbf{lb} \leq \mathbf{Z} \leq \mathbf{ub} \end{aligned} \quad (4.7)$$

The vector of decision variables $\mathbf{Z} \in \mathbb{R}^{(N+1)n_x + Nn_u}$ is organized sequentially to maintain a banded structure in the constraint matrices:

$$\mathbf{Z} = [\mathbf{x}_0^\top \mathbf{u}_0^\top \mathbf{x}_1^\top \mathbf{u}_1^\top \dots \mathbf{u}_{N-1}^\top \mathbf{x}_N^\top]^\top \quad (4.8)$$

The Hessian matrix $\mathbf{H}$ is a large, sparse, block-diagonal matrix containing the state and input weighting matrices $\mathbf{Q}$, $\mathbf{R}$, and the terminal cost $\mathbf{Q}_f$:

$$\mathbf{H} = \text{diag}(\mathbf{Q}, \mathbf{R}, \mathbf{Q}, \mathbf{R}, \dots, \mathbf{R}, \mathbf{Q}_f) \quad (4.9)$$

Correspondingly, the linear cost vector $\mathbf{f}$ accounts for the reference tracking objectives $(\mathbf{x}_{ref}, \mathbf{u}_{ref})$ by expanding the quadratic cost $(\mathbf{x} - \mathbf{x}_r)^\top \mathbf{Q}(\mathbf{x} - \mathbf{x}_r)$:

$$\mathbf{f} = [(-\mathbf{Q}\mathbf{x}_{ref,0})^\top \ (-\mathbf{R}\mathbf{u}_{ref,0})^\top \ \dots \ (-\mathbf{Q}_f\mathbf{x}_{ref,N})^\top]^\top \quad (4.10)$$

The equality constraints $\mathbf{A}_{eq}\mathbf{Z} = \mathbf{b}_{eq}$ encapsulate both the initial state measurement and the discretized LTV dynamics. As implemented in `QPBuilder.m`, the dynamics are rearranged as $-\mathbf{A}_{d,k}\mathbf{x}_k - \mathbf{B}_{d,k}\mathbf{u}_k + \mathbf{I}\mathbf{x}_{k+1} = \mathbf{d}_k$. The resulting system is:

$$\underbrace{\begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ -\mathbf{A}_{d,0} & -\mathbf{B}_{d,0} & \mathbf{I} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & -\mathbf{A}_{d,1} & -\mathbf{B}_{d,1} & \mathbf{I} & \dots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & -\mathbf{A}_{d,N-1} & -\mathbf{B}_{d,N-1} & \mathbf{I} \end{bmatrix}}_{\mathbf{A}_{eq}} \mathbf{Z} = \underbrace{\begin{bmatrix} \mathbf{x}_{meas} \\ \mathbf{d}_0 \\ \mathbf{d}_1 \\ \vdots \\ \mathbf{d}_{N-1} \end{bmatrix}}_{\mathbf{b}_{eq}} \quad (4.11)$$

Implementation Choices The following numerical strategies were adopted in the `MPCController.m` and `QPBuilder.m` classes to ensure real-time feasibility:

- **Sparsity Management:** All matrices are initialized using `spalloc` and con-

structured using sparse blocks (`speye`, `sparse`). This reduces memory footprint and allows the `interior-point-convex` algorithm to perform sparse Cholesky factorizations, significantly accelerating the solve time.

- **Affine Correction:** The vector $\mathbf{b}_{eq}$ includes the affine terms $\mathbf{d}_k$, which represent the difference between the full nonlinear dynamics and the linear approximation. This ensures that even with a simple Euler discretization, the linear model remains "anchored" to the nonlinear plant behavior.
- **Boundary Constraints:** Box constraints on inputs $\mathbf{u}$ and physical state limits (e.g., maximum velocities $\mathbf{v}_{max}$ and yaw rates) are mapped directly to the vectors $\mathbf{lb}$ and $\mathbf{ub}$ based on the index of the decision variables in $\mathbf{Z}$.

SUBSECTION 4.3

Terminal Cost and Terminal Set

To ensure local asymptotic stability and provide a consistent approximation of the infinite-horizon optimal control problem, a quadratic terminal cost is included in the objective function. As discussed in the literature [3], choosing a terminal cost function $V_f(\Delta\mathbf{x}) = \Delta\mathbf{x}^T \mathbf{Q}_f \Delta\mathbf{x}$ that approximates the infinite-horizon cost $J_{0 \rightarrow \infty}$ is essential for stability. In this implementation, the terminal weight matrix $\mathbf{Q}_f$ is designed to act as a Lyapunov candidate for the tracking error dynamics.

In the implemented `TerminalCost.m` class, $\mathbf{Q}_f$ is calculated as the solution $\mathbf{P}$ to the **Discrete Algebraic Riccati Equation (DARE)**. This represents the cost-to-go of an optimal **Discrete Linear Quadratic Regulator (DLQR)** starting from the end of the prediction horizon N .

4.3.1 Hover: Static Equilibrium Linearization

In the stationary `Hover` scenario, the terminal cost is computed once at the start of the simulation. The system is linearized around the equilibrium state $\mathbf{x}_{eq}$ and the gravity-compensating input $\mathbf{u}_{eq}$, resulting in the discrete-time pair $(\mathbf{A}_d, \mathbf{B}_d)$. The terminal matrix is obtained using the MATLAB `dlqr` function:

$$[\mathbf{K}, \mathbf{Q}_f, \mathbf{e}] = \text{dlqr}(\mathbf{A}_d, \mathbf{B}_d, \mathbf{Q}, \mathbf{R}) \quad (4.12)$$

where $\mathbf{Q}$ and $\mathbf{R}$ are the stage cost weighting matrices defined in `ControlParams.m`.

4.3.2 Circle and Spiral: Adaptive Reference Linearization

For trajectories characterized by continuous motion, such as the `Circle` and `Spiral`, a static cost matrix calculated at hover fails to capture the dynamics. To address this, an adaptive strategy is employed where $\mathbf{Q}_f$ is re-evaluated at each time step t . The linearization is performed around the terminal reference state and input:

$$\mathbf{x}_{term} = \mathbf{x}_{ref}(t + N), \quad \mathbf{u}_{term} = \mathbf{u}_{ref}(t + N) \quad (4.13)$$

Solving the DARE at this specific operating point ensures that the terminal cost accurately penalizes deviations from the future reference path.

4.3.3 Stability Verification

To verify the validity of the computed terminal cost, the spectral radius ρ of the closed-loop system is inspected. The solver ensures that all eigenvalues λ_i of the closed-loop transition matrix $(\mathbf{A}_d - \mathbf{B}_d \mathbf{K})$ satisfy the asymptotic stability condition:

$$\rho(\mathbf{A}_d - \mathbf{B}_d \mathbf{K}) = \max_i |\lambda_i| < 1 \quad (4.14)$$

Remark In general, a **terminal set** $\Delta x_{t+p+1} \in \mathcal{X}_{p+1}$ can also be chosen. Chen et al. [4] suggest choosing $\mathcal{X}_{p+1}$ to be a sufficiently small level set of the terminal cost function to guarantee stability.

SECTION 5

Proportional Integral Derivative

This section details the design and implementation of the **Proportional-Integral-Derivative (PID)** controller, which serves as the primary benchmark for evaluating the performance of the proposed LTV-MPC architecture.

SUBSECTION 5.1

Control Architecture

The baseline controller is implemented in `PIDController.m` as a set of four independent **Single-Input Single-Output (SISO)** loops. While the helicopter is a multi-variable system with coupled dynamics, the PID architecture treats the longitudinal position (x), lateral position (y), altitude (z), and yaw heading (Ψ) as decoupled channels. To ensure physical consistency with the helicopter dynamics, where the pitch and roll actuators act on the body-fixed axes, the control law is executed in the **Body Frame** rather than the Inertial Frame.

5.1.1 Error Computation and Shortest-Path Logic

At each sampling instant k , the tracking error in the inertial frame $\mathbf{e}_I(k) \in \mathbb{R}^4$ is computed as the difference between the reference state $\mathbf{x}_r$ generated by the flatness mapping and the measured state $\mathbf{x}_{meas}$:

$$\mathbf{e}_I(k) = \mathbf{x}_{r,1:4}(k) - \mathbf{x}_{meas,1:4}(k) \quad (5.1)$$

Because heading is a periodic variable, a standard subtraction in the yaw channel can lead to non-optimal rotations (e.g., rotating 350° instead of -10°). To enforce **shortest-path behavior**, the yaw error e_Ψ is wrapped into the range $[-\pi, \pi]$ using the `atan2` function:

$$e_{\Psi,wrapped} = \text{atan2}(\sin(e_\Psi), \cos(e_\Psi)) \quad (5.2)$$

5.1.2 Inertial-to-Body Transformation

The translational actuators for pitch (u_x) and roll (u_y) generate forces aligned with the helicopter's local axes. To translate inertial position errors into meaningful actuator commands, the XY error components are projected onto the current Body Frame using the transpose of the rotation matrix $\mathbf{R}_z(\Psi)$:

$$\begin{bmatrix} e_{x,B} \\ e_{y,B} \end{bmatrix} = \begin{bmatrix} \cos(\Psi) & \sin(\Psi) \\ -\sin(\Psi) & \cos(\Psi) \end{bmatrix} \begin{bmatrix} e_{x,I} \\ e_{y,I} \end{bmatrix} \quad (5.3)$$

The vertical (z) and rotational (Ψ) errors are frame-invariant in this model. The resulting error vector used for control is $\mathbf{e}_B(k) = [e_{x,B}, e_{y,B}, e_{z,I}, e_{\Psi,wrapped}]^\top$.

SUBSECTION 5.2

Control Law Implementation

The PID law is implemented in a discrete-time parallel form, utilizing specific numerical approximations for the integral and derivative contributions.

5.2.1 Integral Accumulation and Anti-Windup

The integral state $\mathbf{I}(k)$ is used to eliminate steady-state tracking errors. It is updated via the forward Euler method:

$$\mathbf{I}(k) = \mathbf{I}(k-1) + \mathbf{e}_B(k) \cdot T_s \quad (5.4)$$

To prevent **integrator windup**, a condition where the integral term grows excessively during actuator saturation, a clamping logic is applied. The contribution of the integral term to the control signal is restricted by a threshold $\mathbf{u}_{AW}$ defined in `ControlParams.m`:

$$\mathbf{u}_{int}(k) = \text{sat}(\mathbf{K}_i \odot \mathbf{I}(k), \pm \mathbf{u}_{AW}) \quad (5.5)$$

$$\mathbf{u}_{AW} = [2.0, 2.0, 2.0, 1.0]; \quad (5.6)$$

where $\odot$ denotes the **Element-wise product**. This ensures that the integral action does not destabilize the system when the motors reach their physical limits. The threshold values for the integral terms were selected experimentally.

5.2.2 Discrete Derivative and Gravity Feedforward

The derivative term provides necessary damping to stabilize the inherently unstable open-loop dynamics of the helicopter. It is approximated using a first-order backward difference:

$$\mathbf{u}_{der}(k) = \mathbf{K}_d \odot \frac{\mathbf{e}_B(k) - \mathbf{e}_B(k-1)}{T_s} \quad (5.7)$$

A key feature of the altitude channel is the inclusion of a **model-based gravity feedforward**. Since the vertical dynamics are dominated by gravity, the PID controller is augmented with a constant term u_{ff} representing the thrust required to maintain hover equilibrium:

$$u_{ff} = \frac{g}{b_z}, \quad \mathbf{u}_{FF} = [0, 0, u_{ff}, 0]^\top \quad (5.8)$$

This allows the linear PID loops to operate around a linearized equilibrium point, significantly improving vertical tracking performance.

5.2.3 Final Actuator Command

The total raw control input is the sum of the proportional, integral, derivative, and feedforward components. Before being applied to the plant, the command is passed through a saturation block to respect the normalized actuator limits $\mathcal{U} \in [u_{min}, u_{max}]$:

$$\mathbf{u}(k) = \text{sat}(\mathbf{K}_p \odot \mathbf{e}_B(k) + \mathbf{u}_{int}(k) + \mathbf{u}_{der}(k) + \mathbf{u}_{FF}) \quad (5.9)$$

This architecture provides a robust benchmark baseline, allowing for a direct comparison with the predictive and optimal nature of the MPC.

Results and Discussion

SECTION 6

Experimental Results and Analysis of MPC

In the first part of the experimental results, an in-depth analysis of the results related to the MPC is carried out. To do this, different initial conditions for the helicopter are

PART

IV

used, as well as different sampling times T_s .

The default parameters used for this suite of experiments are faithful to the implementation in the original paper and include the following:

$$Q = Q_f = \text{diag}([50, 50, 50, 10, 3, 3, 1, 2, 15, 15]) \quad (6.1)$$

$$R = \text{diag}([2.0, 2.0, 2.0, 2.0]) \quad (6.2)$$

$$p = 18 \quad (6.3)$$

Remark Terminal cost was not employed for this simulation, setting $Q = Q_f$ as the authors suggest in the original paper. Model Simplification is not used, in order to analyze the complete behavior of the system.

The simulations were conducted using the script `MPCSimulationLoopTs.m`, specifically designed to evaluate the controller's robustness. The analysis focuses on the tracking quality of the two trajectories (Circle and Spiral) while varying the sampling time $T_s \in \{0.02, 0.05, 0.10, 0.20\}$ seconds.

SUBSECTION 6.1

Simulation Method and PW Constant Inputs

A critical aspect of this validation is the consistency to the Linear Time-Varying (LTV) formulation described in the reference paper. The simulation enforces the assumption of **Piecewise (PW) Constant Inputs**. As mentioned in section II, between two sampling instants k and $k + 1$, the control input $\mathbf{u}(t)$ remains constant:

$$\mathbf{u}(t) = \mathbf{u}_k, \quad \forall t \in [kT_s, (k+1)T_s) \quad (6.4)$$

The evolution of the nonlinear helicopter model is simulated by integrating the dynamics using the **Forward Euler** method over the interval T_s . This choice highlights the impact of the approximation: as T_s increases, the assumption that the state derivative $\dot{\mathbf{x}}$ remains constant over the interval, implied by Euler with PW constant inputs, becomes increasingly inaccurate, introducing a mismatch between the predicted LTV model and the actual nonlinear evolution.

Moreover, two distinct initialization strategies were employed:

1. **Start at Origin:** The helicopter is initialized at the state $\mathbf{x}_0 = \mathbf{0}$ (origin), while the reference trajectory begins at a distinct operating point (e.g., altitude $z = 0.5\text{m}$ for Spiral and $z = 1.0\text{m}$ for Circle). The initial linearization is performed using the origin as the nominal trajectory and adopting the warm-start strategy in the next steps. This scenario tests the MPC's ability to handle large initial errors and nonlinearities.
2. **Perturbation (Deviation):** The helicopter is initialized with a small perturbation (0.15m on x-axis) from the first point of the desired trajectory. The initial linearization uses the analytical reference generated via differential flatness as nominal trajectory. This scenario evaluates steady-state tracking performance.

SUBSECTION 6.2

Sampling Time and Initial Condition Impact

The sampling time T_s plays a dual role: it defines the frequency of control updates and the validity window of the LTV approximation. Small sampling times ($T_s = 0.02\text{s}$) result in frequent linearizations, ensuring that the Jacobian matrices A_k, B_k accurately

reflect the local dynamics. Conversely, larger sampling times ($T_s \geq 0.10\text{s}$) extend the prediction horizon in time but degrade the accuracy of the intra-step prediction.

6.2.1 Circle Trajectory Results

The impact of the sampling time on the LTV-MPC performance is visually demonstrated in Figure 5.

In the **Left figure** scenario, the controller utilizing $T_s = 0.02\text{s}$ (blue line) executes a smooth, optimal intercept motion to reach the circular reference. In contrast, the trajectory for $T_s = 0.20\text{s}$ (purple line) is notably segmented. This error occurs because the Linear Time-Varying (LTV) approximation loses its validity during the long time gap between updates. The linear model calculated at the start of the step fails to predict the complex behavior of the helicopter for the full 0.20 seconds. Consequently, the controller applies a constant input based on a prediction that quickly becomes outdated as the helicopter moves, preventing the smooth turning motion required to follow the reference.

The **Right figure** shows the dynamics where the helicopter is initialized with a small perturbation from the first point of the trajectory. While the high-frequency controllers ($T_s \leq 0.05\text{s}$) tend to follow the trajectory, the simulation with $T_s = 0.20\text{s}$ (purple line) reveals a critical failure in tracking the curvature.

Instead of maintaining the circular path, the helicopter drifts significantly inside the circle, tracing a distorted trajectory with a much smaller radius. This occurs because the linearized model, computed at the beginning of the 0.20s interval, cannot capture the continuous evolution of the helicopter orientation required to keep the turn. The controller effectively predicts a path that is too straight for the duration of the hold, causing the helicopter to consistently fall away from the reference arc before the next update can compensate.

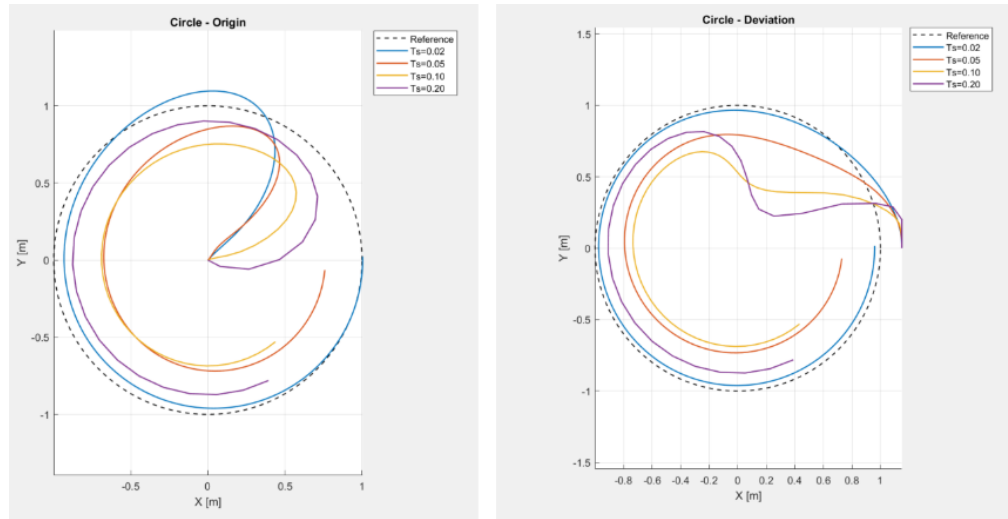


Figure 5. Trajectory tracking performance for the Circle scenario in the XY-plane. **Left:** Recovery from the Origin ($\mathbf{x}_0 = \mathbf{0}$). **Right:** Recovery from an initial deviation near the reference. The degradation of the LTV approximation is visually distinct as T_s increases from 0.02s (blue) to 0.20s (purple).

6.2.2 Spiral Trajectory Results

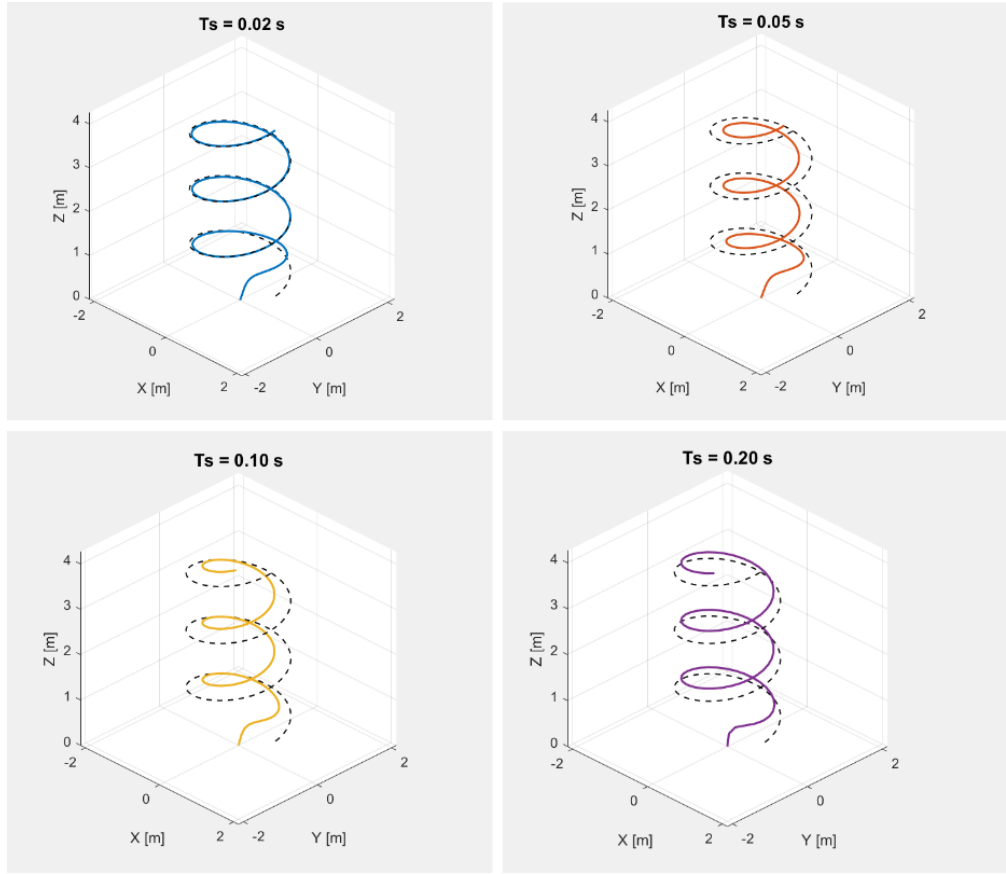


Figure 6. Trajectory tracking performance for the Spiral scenario with origin as starting point for the helicopter.

Figure 6 presents the tracking performance for the 3D Spiral scenario with the helicopter initialized at the origin ($\mathbf{x}_0 = \mathbf{0}$).

- At the highest sampling frequency ($T_s = 0.02s$), the MPC tracks the reference with no delay. The helicopter remains almost perfectly adjacent to the desired trajectory for the entire simulation, handling the initial climb and turn without separating from the path. Even at $T_s = 0.05s$, the tracking remains robust with negligible lag but with a highest error, demonstrating that the LTV model is updated frequently enough to predict the motion.
- As the sampling time increases, a significant delay appears. For $T_s = 0.10s$ and especially $T_s = 0.20s$, the helicopter appears late compared to the reference. It constantly chases the target point but remains visibly behind it. This results in a spatially shifted trajectory, unable to close the gap with the ideal path.

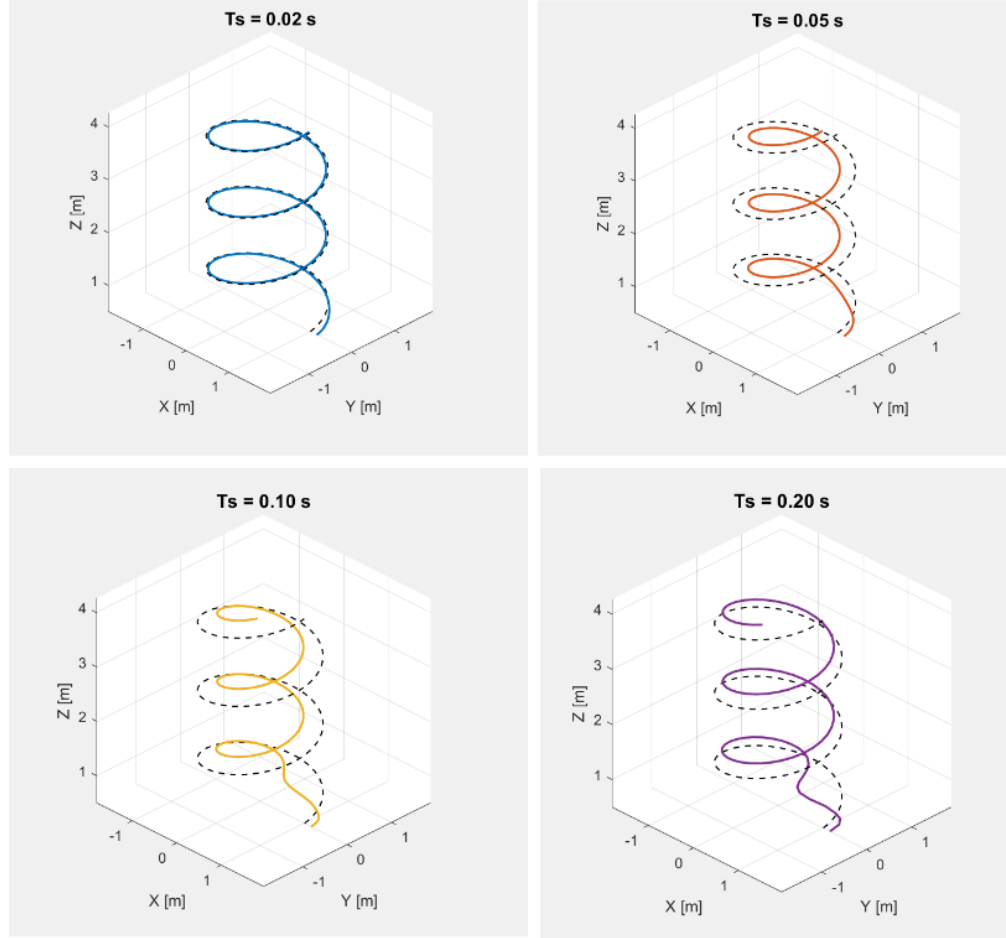


Figure 7. Trajectory tracking performance for the Spiral scenario with initial perturbation on the reference trajectory as starting point for the helicopter.

Figure 7 illustrates the results when the helicopter is initialized with a lateral offset of 15 cm from the reference Spiral.

- For a sampling time of $T_s = 0.02$ s, the controller shows very low delay and almost perfect tracking performance. The helicopter quickly converges to the reference trajectory with negligible tracking error.
- For $T_s = 0.05$ s, although no evident delay is observed, the tracking error is higher compared to the smaller sampling time, resulting in a less accurate trajectory tracking.
- For slower sampling times, the increased control delay leads to a degradation in performance. In particular, for $T_s = 0.10$ s and 0.20 s, the delayed control action prevents proper convergence to the spiral, causing the trajectories to remain inside the reference curve.

6.2.3 Position Error Analysis

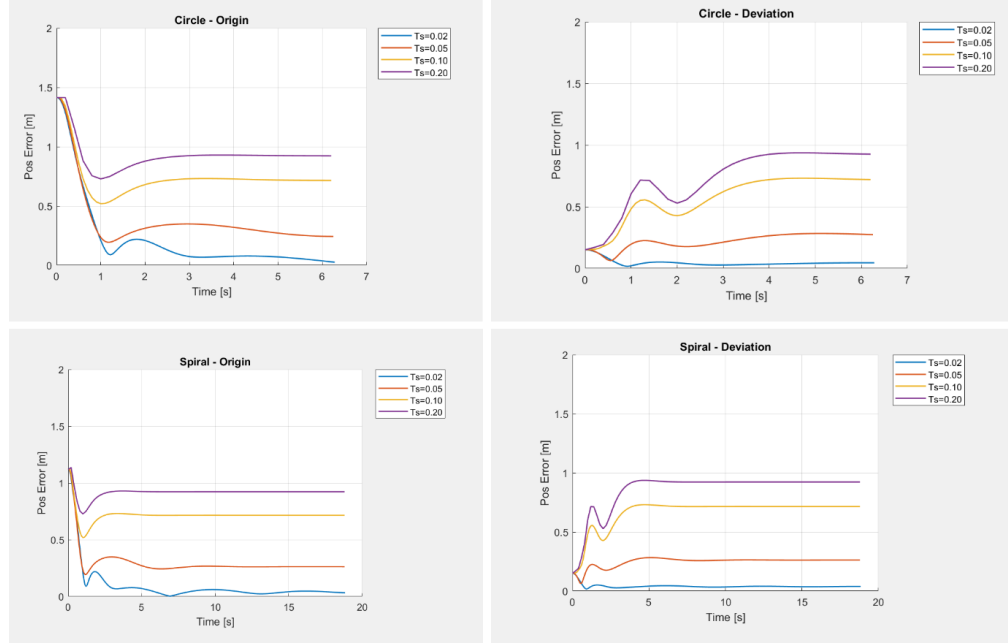


Figure 8. Position Error evolution during the simulation for the proposed scenarios and both shapes.

Figure 8 summarizes the evolution of the position tracking error over time for both the Circle and Spiral trajectories, considering different sampling times T_s and the two initialization conditions. The plots report the Euclidean position error $\|\mathbf{p} - \mathbf{p}_{ref}\|$ as a function of simulation time. In this context position $\mathbf{p}$ indicates the position of the helicopter with respect to the inertial frame $[x_I, y_I, z_I]$.

For small sampling times ($T_s = 0.02\text{s}$ and 0.05s), the error rapidly decreases after the transient phase and converges to a very small value.

As the sampling time increases ($T_s = 0.10\text{s}$ and 0.20s), the steady-state tracking error becomes significantly larger. However, an important observation is that the error does not diverge or grow unbounded over time. Instead, it converges to an approximately constant value that depends on the sampling time.

Higher sampling times reduce tracking accuracy due to delayed updates and poorer intra-step predictions, but the closed-loop MPC structure prevents error explosion. The system reaches a stable error level in all cases and this error grows larger as the sampling time increases.

Table 1. Circle Scenario: MSE and RMSE of Position

Start Condition	Ts	MSE Position	RMSE Position
Origin	0.02	0.1478	0.3844
Perturbation	0.02	0.00274	0.05236
Origin	0.05	0.2163	0.4651
Perturbation	0.05	0.05240	0.2289
Origin	0.10	0.5777	0.7601
Perturbation	0.10	0.3565	0.5971
Origin	0.20	0.8896	0.9432
Perturbation	0.20	0.5829	0.7635

Table 2. Spiral Scenario: MSE and RMSE of Position

Start Condition	Ts	MSE Position	RMSE Position
Origin	0.02	0.03781	0.1944
Perturbation	0.02	0.001888	0.04345
Origin	0.05	0.1039	0.3223
Perturbation	0.05	0.06331	0.2516
Origin	0.10	0.5183	0.7199
Perturbation	0.10	0.4606	0.6787
Origin	0.20	0.8443	0.9188
Perturbation	0.20	0.7612	0.8725

The tables above summarize the **Mean Squared Error (MSE)** and **Root Mean Squared Error (RMSE)** of the position for the Circle and Spiral scenarios, under two different start conditions: *Origin* and *Perturbation*. The definition of these evaluation metrics is reported in section 7.2.

A clear trend emerges with respect to the sampling time T_s : as T_s increases, both MSE and RMSE grow consistently in all scenarios and starting conditions. This indicates that larger sampling intervals reduce the accuracy of the system. The initial condition also has a significant effect on the error. When the system starts at the *Origin*, the errors are generally higher, especially for small T_s , compared to the *Perturbation* case. This can be attributed to the initial trajectory reaching phase, during which a significant error accumulates until the system converges to the reference path. In contrast, starting from a *Perturbation* closer to the reference results in smaller errors, particularly noticeable at low T_s .

Overall, the results highlight the combined impact of sampling time and initial condition on the position accuracy. Smaller T_s and different initial conditions significantly improve the predictive performance of the model.

6.2.4 Linearization Error Analysis

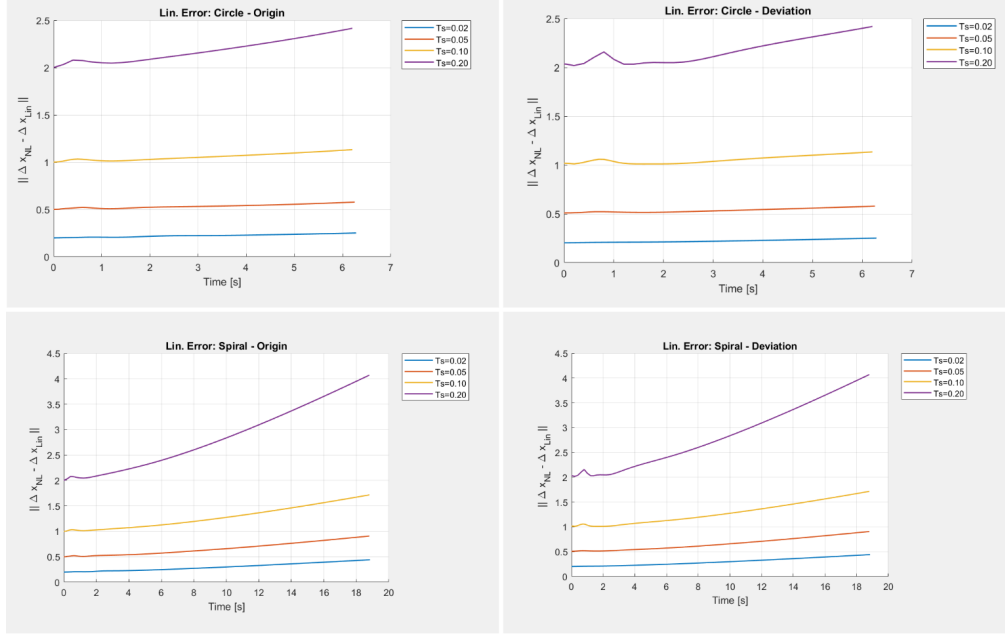


Figure 9. Linearization error during the simulation for the proposed scenarios and both shapes.

Figure 9 shows the evolution of the linearization error during the simulations for the Circle and Spiral scenarios, considering both *Origin* and *Perturbation* start conditions. The linearization error is measured as the norm of the difference between the true nonlinear system and the linearized approximation, scaled by the sampling time:

$$T_s \|f(\hat{\mathbf{x}}_k, \hat{\mathbf{u}}_k) - (A_k \hat{\mathbf{x}}_k + B_k \hat{\mathbf{u}}_k)\| \quad (6.5)$$

As the sampling time T_s increases, the validity of the linearization deteriorates more rapidly. Under the assumption of a piecewise-constant (PW-constant) input, the linearized model remains accurate only at the beginning of each sampling interval, while deviations progressively grow within the interval. This behavior leads to a cumulative effect, where the linearization error increases over time due to the reduced update frequency of the linear model. On the other hand, for smaller sampling times, the linearization is performed more frequently, allowing the linear model to better capture the system dynamics. As a result, the linearization error accumulates more slowly during the simulation.

SECTION 7

Comparative Study: MPC against PID

This section presents a comparative performance analysis of the proposed control strategies across the defined flight scenarios. The LTV-MPC architecture is benchmarked against the baseline PID controller to evaluate tracking precision and robustness. Quantitative metrics, including Root Mean Squared Error (RMSE) and average computational solve times, are employed to validate the real-time feasibility and tracking capabilities of the predictive approach. The control framework and non-linear simulations were implemented in MATLAB R2024b on an Ubuntu 24.04 LTS workstation equipped with an Intel Core i7 13650HX CPU and 32 GB of DDR5 RAM. Comprehensive data analysis and high-fidelity visualizations were subsequently performed within a Python 3.10.0 environment to ensure rigorous assessment of the results.

SUBSECTION 7.1

Comparison Criteria

To ensure a fair and comprehensive comparison between the two control algorithms, several common evaluation strategies have been adopted. Both controllers share the same initial condition, with the helicopter initialized at the beginning of the reference trajectory. In addition, two different parameter configurations are considered: different weighting matrices Q , Q_f , and R for the MPC controller, and distinct tuning strategies for the PID controller through the gains K_p , K_i , and K_d . For the MPC approach, the initial linearization is performed along the nominal reference trajectory obtained via differential flatness mapping, ensuring consistency in the model initialization and reference tracking objectives. Both trajectory shapes and the hover have been evaluated, and the corresponding results are summarized in a final table, providing a quantitative comparison of the control performance.

SUBSECTION 7.2

Evaluation Metrics

To quantitatively assess the performance, several evaluation metrics are considered. These metrics capture both tracking accuracy and computational efficiency.

The tracking performance is evaluated using the **Mean Squared Error (MSE)** and the **Root Mean Squared Error (RMSE)** of the system states with respect to the reference trajectory. Let $\mathbf{p} = (x_I, y_I, z_I, \Psi)$ denote the system state vector in the inertial frame and $\mathbf{p}_{\text{ref}}$ the corresponding reference trajectory. The state MSE is defined as:

$$\text{MSE}_{\text{state}} = \frac{1}{N} \sum_{k=1}^N \|\mathbf{p}_k - \mathbf{p}_{\text{ref},k}\|^2,$$

where $N = \frac{T_{\text{simulation}}}{T_s}$ is the total number of simulation steps. This metric provides a measure of the average squared deviation from the reference trajectory. The RMSE is then computed as:

$$\text{RMSE}_{\text{state}} = \sqrt{\text{MSE}_{\text{state}}},$$

offering a more interpretable measure expressed in the same physical units as the states (meters and radians).

In addition, the control effort is assessed through the **Input MSE**, which reflects the magnitude of the actuation required by each controller during trajectory tracking.

Let $\mathbf{u} = (u_x, u_y, u_z, u_\Psi)$ denote the control input vector and $\mathbf{u}_{\text{ref}}$ the corresponding reference input, reconstructed using the differential flatness. The Input MSE is defined as:

$$\text{MSE}_{\text{input}} = \frac{1}{N} \sum_{k=1}^N \|\mathbf{u}_k - \mathbf{u}_{\text{ref},k}\|^2.$$

The above metrics are calculated in `MPCSimulationLoop.m` and `PIDSimulationLoop.m`, respectively.

Finally, the **Average Solver Time** is reported for both control strategies and scenarios to evaluate their computational cost and real-time feasibility. For the MPC controller, This metric corresponds to the average time required to execute the entire MPC pipeline at each receding horizon step, including problem setup and the solution of the constrained optimization problem. In contrast, for the PID controller, the solver time corresponds to the time required to compute the control input using the pre-determined closed-loop gain.

These metrics enable a fair and comprehensive comparison of PID and MPC performance in both trajectory tracking and hover scenarios.

SUBSECTION 7.3

MPC Configurations

The MPC controller parameters are centralized in `ControlParams.m`. To maintain consistency with standard RC communication rates and real-time hardware constraints in this comparison, the sampling time is fixed to $T_s = 20$ ms (50 Hz). The prediction horizon is chosen as $p = 18$, which provides a look-ahead window of 0.36 s, sufficient for anticipating the dynamics of the trajectories considered. To evaluate the versatility of the LTV-MPC architecture, two distinct simulation and tuning strategies were implemented, characterized by different weighting matrices and model fidelity settings.

Moreover, to ensure a high-fidelity representation of the closed-loop system behavior, the state propagation step is performed using the `ode45` solver. Specifically, the continuous-time nonlinear dynamics of the helicopter are integrated over each sampling interval $[t_k, t_k + T_s]$, assuming that the optimal control input computed by the MPC is applied as a **Piecewise Constant (PW)** signal during the step.

7.3.1 Paper-Fidelity Configuration

This configuration aims to replicate the experimental conditions described in the original work by Kunz et al. In this mode, the controller utilizes a simplified linearization (`simplify_coupling_flag = true`) where cross-coupling terms between the rotational states ($\Psi, \dot{\Psi}$) and translational accelerations are suppressed to improve experimental robustness. The weights are chosen to allow a smooth control action:

$$Q = Q_f = \text{diag}([50, 50, 5, 10, 3, 3, 1, 2, 15, 15]) \quad (7.1)$$

$$R = \text{diag}([2.0, 2.0, 2.0, 2.0]) \quad (7.2)$$

Terminal cost was not employed for this simulation, setting $Q = Q_f$ as the authors suggest in the original paper.

7.3.2 High-Performance Configuration

This strategy is designed to push the tracking limits of the miniature helicopter. The full physics model is enabled within the controller's Jacobian engine (`simplify_coupling_flag = false`), allowing the MPC to exploit the nonlinear couplings for better prediction.

The state weighting matrix Q is increased by an order of magnitude on position errors to enforce aggressive tracking, while R is reduced to allow high actuator authority:

$$Q = Q_f = \text{diag}([500, 500, 50, 10, 3, 3, 1, 2, 15, 15]) \quad (7.3)$$

$$R = \text{diag}([0.1, 0.1, 0.1, 0.1]) \quad (7.4)$$

Also in this simulation, terminal cost was not employed.

7.3.3 Terminal Cost Configuration

By reproposing the same configurations presented for the High-Performance Configuration, the terminal cost has been enabled, calculated with reference to what is explained in section 4.3.

SUBSECTION 7.4

PID Tuning Strategies

The performance evaluation is orchestrated by the `PIDSimulationLoop.m` script, which simulates the closed-loop behavior of the helicopter. The simulation proceeds in discrete time steps $t_k = k \cdot T_s$. At each iteration, the loop executes a three-stage sequence:

1. **Reference Generation:** A reference is generated following the differential flatness approach used in the MPC.
2. **Control Law Evaluation:** The PID controller computes the control inputs based on the instantaneous tracking error.
3. **Numerical Plant Propagation:** To accurately model the continuous-time physics of the helicopter, the state evolution between control updates is solved numerically. The simulation employs the `ode45` solver to integrate the nonlinear equations of motion over the interval $[t_k, t_k + T_s]$. During this integration sub-step, the control input $\mathbf{u}$ is held constant.

7.4.1 Standard Tuning

The standard tuning set represents a conservative approach designed for robust regulation and steady-state stability. These gains are a starting point for the PID configuration:

- $\mathbf{K}_p = [2.0, 2.0, 5.0, 2.0]^\top$
- $\mathbf{K}_i = [0.5, 0.5, 2.0, 0.1]^\top$
- $\mathbf{K}_d = [1.5, 1.5, 2.5, 0.5]^\top$

Higher gain values were assigned to the z -component to provide sufficient authority for gravity compensation. For the horizontal plane, the x and y controllers utilize equivalent gains, assuming symmetric behavior in the longitudinal and lateral axes. For motion in the horizontal plane, identical gain values are applied to the x and y controllers, reflecting the assumed symmetry between the longitudinal and lateral dynamics.

7.4.2 High-Performance Tuning

A high-performance configuration was established via manual trial-and-error, adjusting the gains to push the PID architecture to its stability limits, and it is defined by the following vectors:

- $\mathbf{K}_p = [6.0, 6.0, 10.0, 3.0]^\top$

- $\mathbf{K}_i = [3.5, 3.5, 8.0, 0.5]^\top$
- $\mathbf{K}_d = [3.0, 3.0, 5.0, 1.0]^\top$

The deployment of these two strategies ensures that the LTV-MPC is benchmarked against a PID architecture pushed to its stability limits.

SUBSECTION 7.5

MPC Results

7.5.1 Circle Trajectory

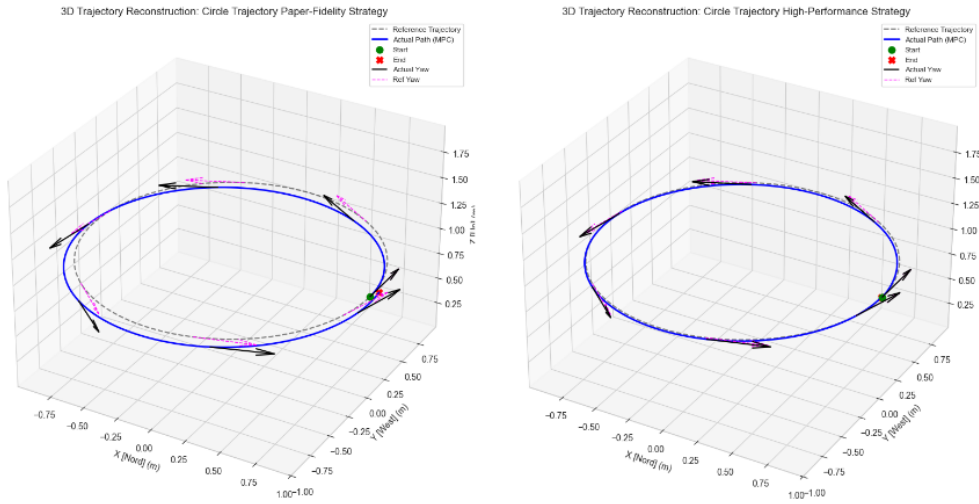


Figure 10. Circle Trajectory Results for MPC.

The Circular trajectory results, presented in the figure above, highlight the impact of controller tuning and model fidelity on tracking performance.

Paper-Fidelity Strategy (a) This simulation replicates the parameters defined in the reference study. The controller exhibits a stable but conservative behavior. Because the input weighting matrix R is relatively large compared to the state weighting Q , the MPC solver adopts a energy-efficient solution, cutting the corner of the circle to minimize control effort.

High-Performance Strategy (b) By transitioning to a high-performance configuration, the tracking error is significantly reduced. The state error weights Q were increased to prioritize position accuracy, while the input penalty R was decreased to allow more aggressive actuator responses. Moreover, by disabling the yaw-coupling simplification, the LTV model provides a more accurate prediction of the lateral forces required to sustain the circular motion.

7.5.2 Spiral Trajectory

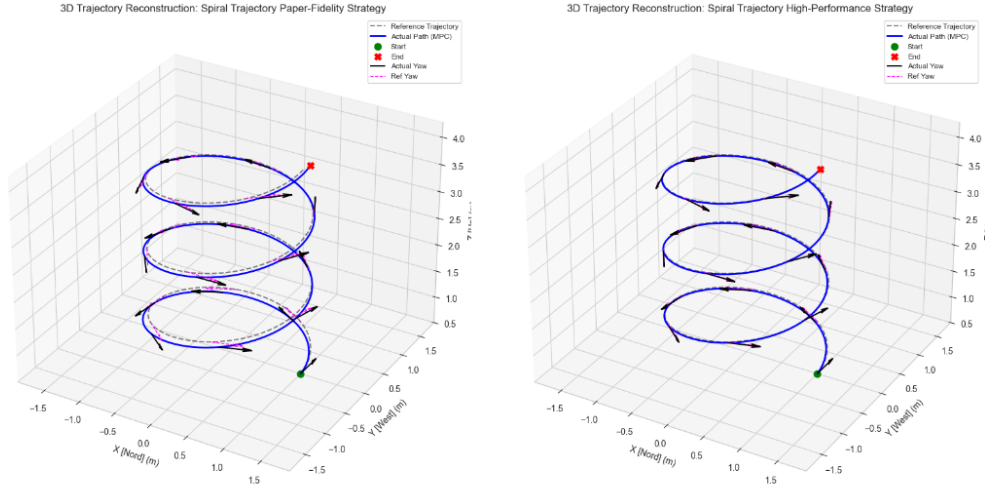


Figure 11. Spiral Trajectory Results for MPC.

Paper-Fidelity Strategy (a) In the conservative configuration, the controller maintains a stable helical path. This behavior is primarily due to the high input penalty ($R = 2.0$), which discourages the controller from applying the aggressive pitch and roll maneuvers required to perfectly maintain a 1-meter radius while climbing. The simplified model used in this strategy provides high robustness against unmodeled dynamics at the cost of a visible tracking lag, particularly in the heading vectors which tend to point slightly inside the trajectory curve.

High-Performance Strategy (b) The transition to the high-performance strategy results in a significant increase in tracking fidelity. By utilizing the full physical model (disabling the yaw-coupling simplification) and aggressive weighting ($Q_{pos} = 500$, $R = 0.1$), the actual path becomes nearly indistinguishable from the reference trajectory. The controller effectively manages the 3D centripetal forces and the vertical climb rate, demonstrating that the LTV-MPC can exploit the identified nonlinear dynamics to minimize phase lag.

7.5.3 Temporal Tracking Performance

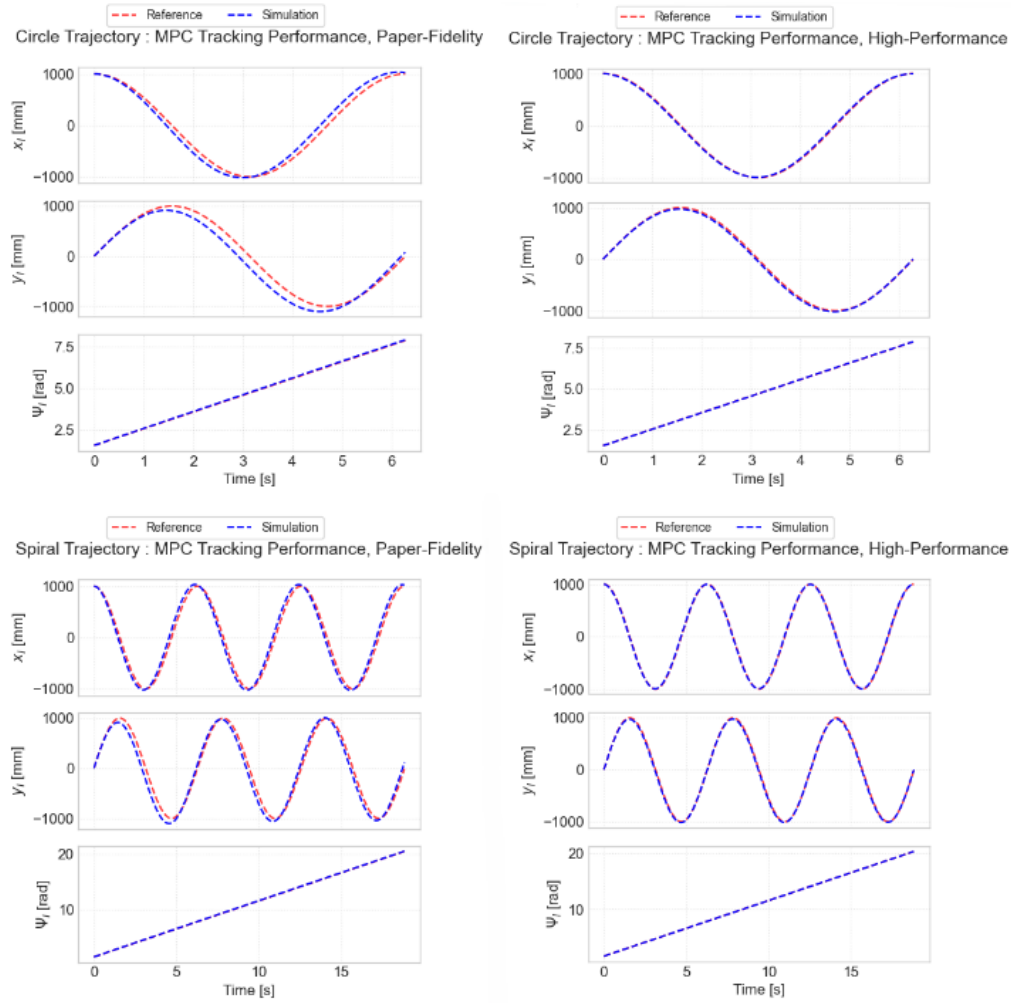


Figure 12. The temporal evolution of the inertial states for the **Circle** and **Spiral** trajectories. The 2x2 grid compares the Paper-Fidelity strategy (left) against the High-Performance configuration (right).

Paper-Fidelity Parameter Analysis In the left column, the controller prioritized input smoothness over tracking precision. This resulted in a slightly *phase delay* and an amplitude mismatch in the oscillatory components of the position. Such behavior is expected when the input weighting matrix R highly penalizes the applied inputs.

High-Performance Parameter Analysis In the right column, the implementation of a high-performance tuning and the use of the full physical model enabled the plant to track the reference trajectories with negligible error. The simulation curves for x_I , y_I , and Ψ show almost perfect synchronization with the reference commands.

SUBSECTION 7.6

PID Results

7.6.1 Circle Trajectory

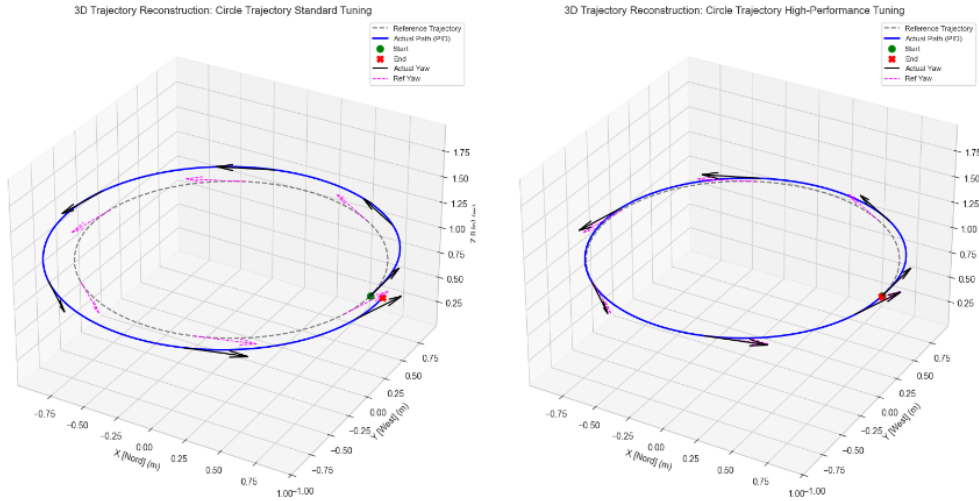


Figure 13. Circle Trajectory Results for PID.

Standard Tuning (a) As shown in the left plot, the standard tuning configuration results in poor tracking fidelity during the circular trajectory. The helicopter exhibits a substantial offset, effectively flying a circle with a radius significantly larger than the commanded 1.0 m.

High-Performance Tuning (b) The right plot in illustrates the results for the high-performance tuning. The increased gains allow the controller to counteract the system's inertia more effectively, resulting in a path that lies much closer to the reference trajectory. Although the position error is reduced compared to the standard case, the PID architecture still demonstrates a high sensitivity to the nonlinearities of the helicopter model.

7.6.2 Spiral Trajectory

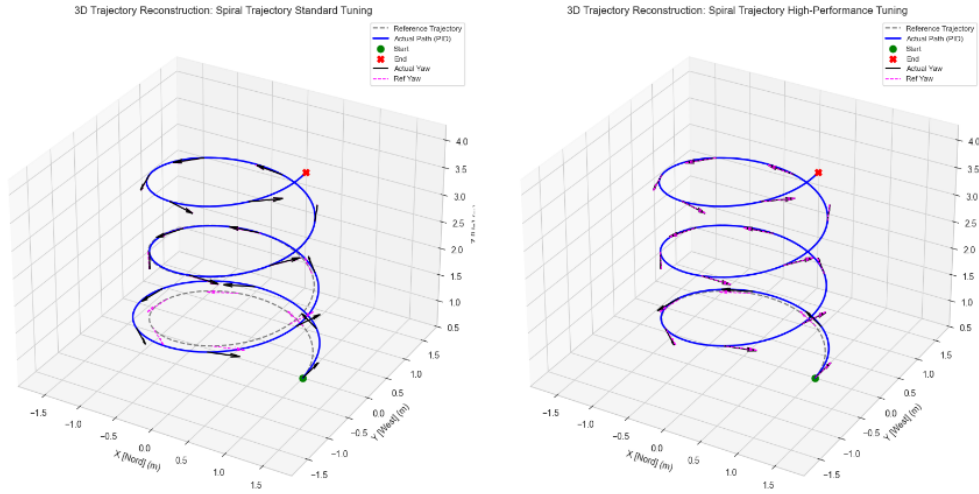


Figure 14. Spiral Trajectory Results for PID.

Standard Tuning (a) In the standard tuning configuration (left plot), the helicopter exhibits a significant offset that persists throughout the first revolution. This steady-state error is a direct result of the controller’s inability to compensate for the centripetal forces required by the 1.0 m radius path while simultaneously managing the vertical climb.

High-Performance Tuning (b) The right plot, displays the results for the high-performance tuning. The increased proportional and integral gains provide the necessary authority to reduce the tracking lag and pull the actual path closer to the reference trajectory. While the spatial fidelity is markedly improved compared to the standard case, the PID architecture still lacks the model-based predictive capability of the MPC.

SUBSECTION 7.7

Overall Comparison

This section evaluates the performance of the implemented control strategies through a comparative analysis of tracking accuracy, control effort, and computational feasibility. The quantitative results obtained for the three flight scenarios (**Hover**, **Circle**, and **Spiral**) are summarized in Tables 3 and 4.

7.7.1 Trajectory Tracking Performance

The experimental data indicates that all controllers achieve high-fidelity regulation during the **Hover** task, with errors reaching the numerical precision limits of the simulation environment. However, significant performance gaps emerge during trajectory simulation.

The **High-Performance MPC** configuration proves the highest tracking precision, achieving a State RMSE of approximately 33 mm for the circular path and 31 mm for the spiral. While the **High-Performance PID** is capable of reaching comparable spatial accuracy (33–51 mm), it requires significantly more aggressive actuator activity. This is proved by the Input MSE metrics: the high-performance PID exhibits a control effort nearly three orders of magnitude higher than the MPC (~ 0.8 vs ~ 0.001). This suggests

that the PID matches the MPC's precision only by operating near the saturation limits, which would likely induce mechanical vibration and actuator stress in a physical testbed.

7.7.2 Comparing Controller Performance

The primary advantage of the LTV-MPC architecture is its ability to exploit the internal model to anticipate future reference changes. In the **Paper-Fidelity** configuration, the MPC maintains a smooth and stable trajectory even with conservative weights. In contrast, the **Standard PID** suffers from a characteristic phase lag during curvilinear flight, as it lacks the predictive terms necessary to compensate for centripetal forces.

From a computational standpoint, the PID solver is extremely efficient, with an average solve time of approximately 6 μ s. The MPC, requiring the solution of a sparse Quadratic Program at each step, exhibits a solve time between 3.8 and 4.3 ms. Given the 20 ms sampling period (50 Hz), the MPC utilizes only 20% of the available computational window, confirming its suitability for real-time applications on modern embedded hardware.

Table 3. Quantitative Performance Metrics for MPC Strategies

Scenario	State MSE	State RMSE [m]	Input MSE	Avg. Solve Time [ms]
<i>MPC Paper-Fidelity</i>				
Hover	7.79×10^{-20}	2.79×10^{-10}	1.11×10^{-20}	4.28
Circle	0.0290	0.1703	4.16×10^{-3}	3.87
Spiral	0.0243	0.1560	3.53×10^{-3}	3.82
<i>MPC High-Performance</i>				
Hover	6.09×10^{-20}	2.47×10^{-10}	3.97×10^{-20}	4.26
Circle	0.0011	0.0336	1.23×10^{-3}	4.33
Spiral	0.0010	0.0316	0.51×10^{-3}	4.13

Table 4. Quantitative Performance Metrics for PID Strategies

Scenario	State MSE	State RMSE [m]	Input MSE	Avg. Solve Time [ms]
<i>PID Standard Tuning</i>				
Hover	0.0000	0.0000	0.0000	0.026
Circle	0.0375	0.1938	0.0396	0.006
Spiral	0.0130	0.1140	0.0139	0.007
<i>PID High-Performance Tuning</i>				
Hover	0.0000	0.0000	0.0000	0.053
Circle	0.0027	0.0515	0.7445	0.006
Spiral	0.0011	0.0337	0.8580	0.006

SUBSECTION 7.8

Impact of Terminal Cost

To assess the mathematical robustness of the LTV-MPC, a simulation was conducted using the **Adaptive Terminal Cost** strategy. In this configuration, the terminal weight matrix $\mathbf{Q}_f$ is not assumed equal to the stage cost $\mathbf{Q}$, but is instead calculated at each sampling instant by solving the Discrete Algebraic Riccati Equation (DARE) based on the terminal reference point of the horizon. The quantitative results are summarized in Table 5.

7.8.1 Analysis of Tracking Performance Trade-offs

Despite the theoretical advantages, the results show a significant increase in the State RMSE for the **Circle** (0.3501 m) and **Spiral** (0.3342 m) trajectories compared to the standard High-Performance case. This performance degradation is primarily driven by **Numerical Dominance**: the DARE solution **P** typically results in weighting values that are several orders of magnitude higher than the stage cost **Q**.

For this reason, the optimizer becomes overly sensitive to the error at the final step of the prediction horizon. With a relatively short horizon ($p = 18$, or approximately 0.36 s), the controller focuses on reaching the terminal reference point rather than accurately following the path along earlier steps. As a result, the control action becomes stiff.

Table 5. MPC Metrics with Terminal Cost (DARE)

Scenario	State MSE	State RMSE [m]	Input MSE	Avg. Solve Time [ms]
Hover	6.72×10^{-20}	2.59×10^{-10}	4.62×10^{-20}	4.48
Circle	0.1226	0.3501	0.1012	5.61
Spiral	0.1117	0.3342	0.0550	5.93

Conclusions

SECTION 8

Conclusions and Future Work

This work successfully replicates and extends the fast LTV-MPC framework for miniature helicopters, providing a faithful MATLAB implementation that reproduces the qualitative behavior and quantitative performance reported in the reference paper. The main contributions include a modular flatness-based trajectory generation, an efficient sparse LTV-MPC formulation with warm-starting, and a systematic comparison against a well-tuned PID baseline. Furthermore, an in-depth analysis of the sampling time T_s and initial conditions was performed to assess the robustness of the linearization approach. Across all dynamic scenarios, MPC consistently outperforms PID by achieving significantly lower tracking errors with orders-of-magnitude less control effort, highlighting the advantage of prediction and constraint handling over purely reactive control. The control objectives are fully accomplished, as the helicopter accurately tracks hover, circular, and spiral trajectories, confirming the robustness and high fidelity of the proposed replication. While the inclusion of a terminal cost derived from the DARE ensures theoretical stability, its adaptive formulation leads to degraded tracking performance for short prediction horizons, revealing a practical trade-off between stability guarantees and path-following accuracy. Future work will focus on integrating an **Extended Kalman Filter (EKF)** for realistic state estimation, mitigating terminal cost over-penalization, and deploying the controller on embedded onboard hardware to assess real-time flight performance.

PART

V

References

- [1] K. Kunz, S. M. Huck, and T. H. Summers, “Fast model predictive control of miniature helicopters,” in *Proceedings of the 2013 European Control Conference (ECC)*, (Zürich, Switzerland), pp. 1377–1382, IEEE, 2013.
- [2] R. M. Murray, “Optimization based control,” in *Proceedings of the 10th European Control Conference (ECC)*, p. 1065, 2010. Accessed: 2026-01-19.
- [3] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, “Constrained model predictive control: Stability and optimality,” *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.
- [4] H. Chen and F. Allgöwer, “A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability,” *Automatica*, vol. 34, no. 10, pp. 1205–1217, 1998.